

Evaluation and Optimization of Multicore Performance Bottlenecks in Supercomputing Applications

Jeff Diamond¹, Martin Burtscher²,
John D. McCalpin³, Byoung-Do Kim³,
Stephen W. Keckler^{1,4}, James C. Browne¹

¹University of Texas, ²Texas State, ³Texas Advanced Computing Center, ⁴NVIDIA

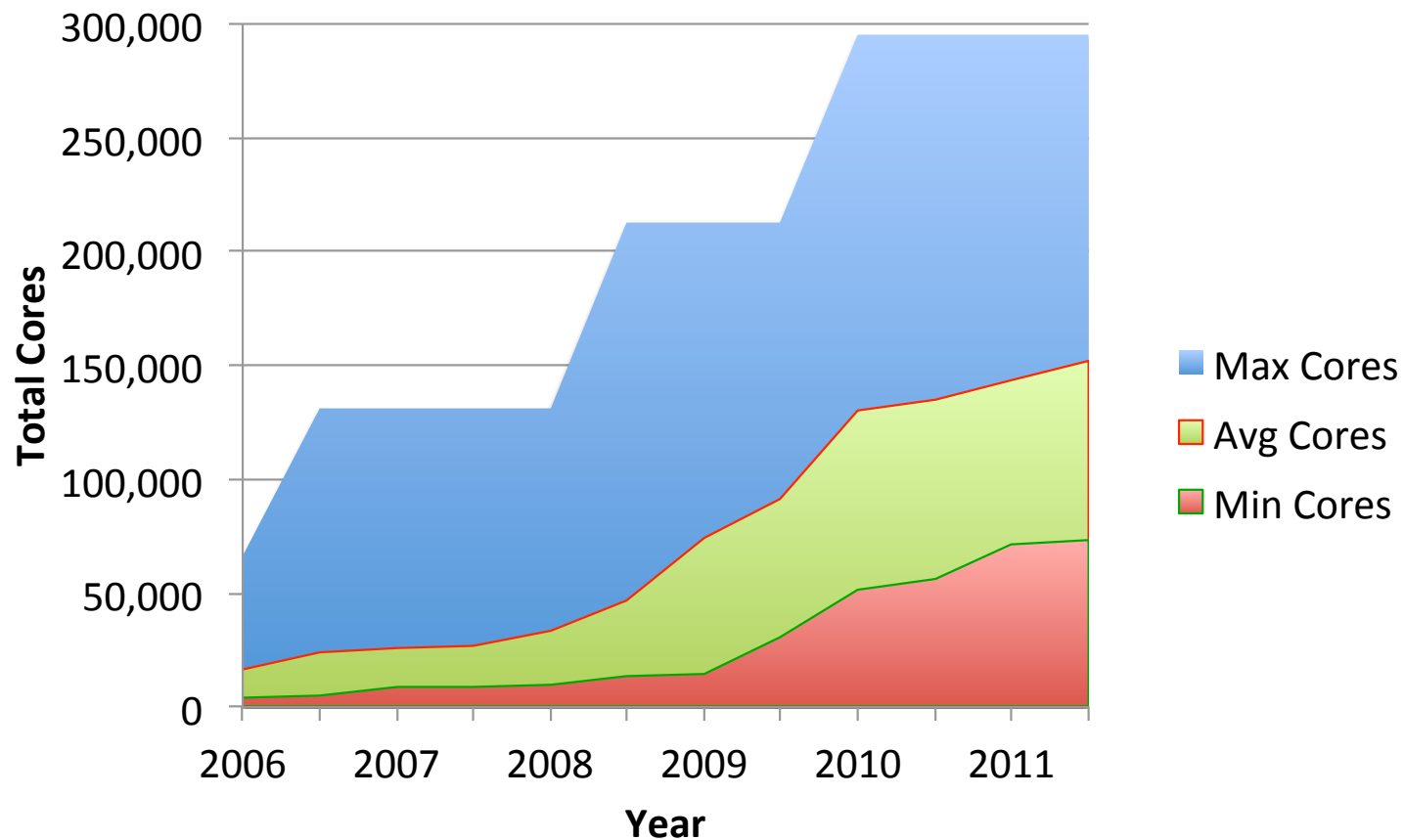


THE UNIVERSITY OF TEXAS AT AUSTIN

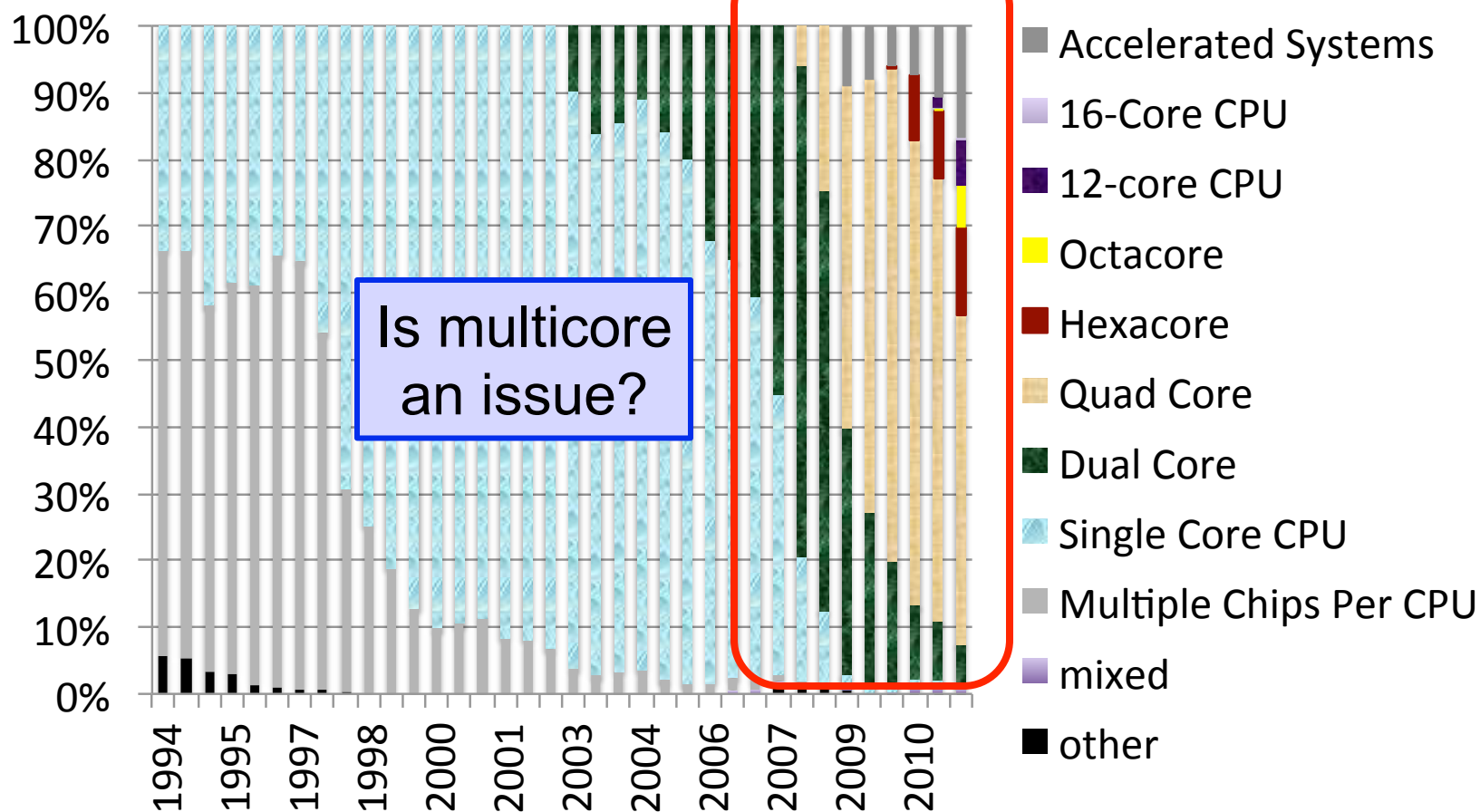


Trends In Supercomputers

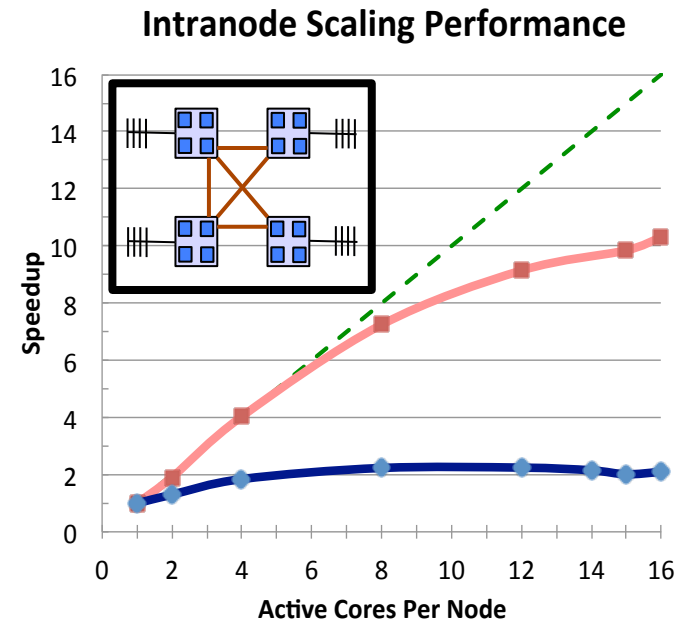
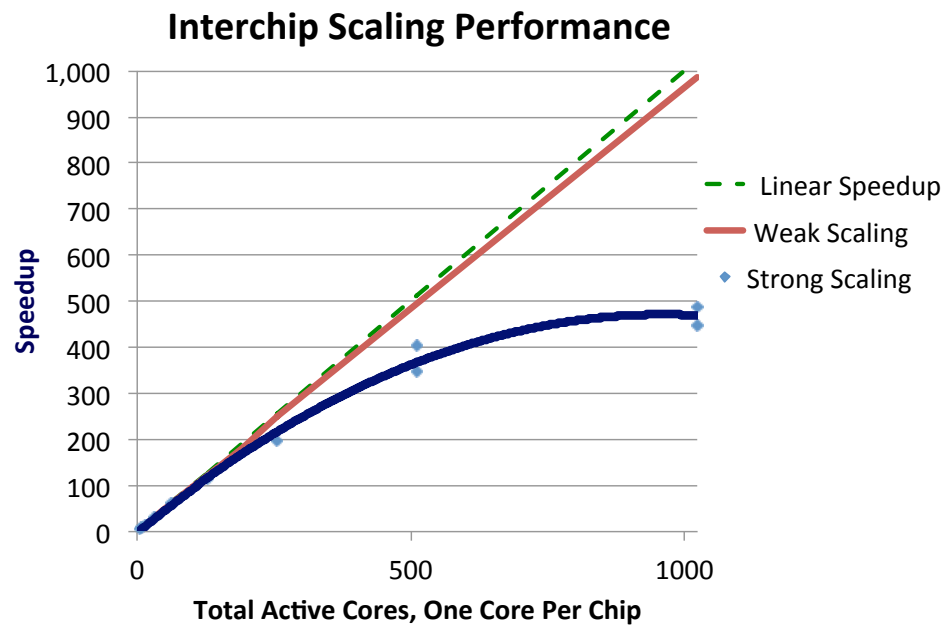
Top Ten Supercomputers - Core Count



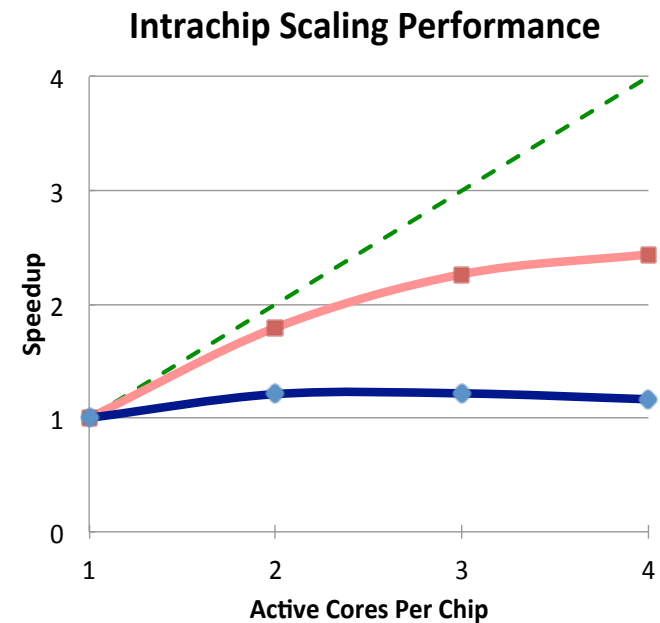
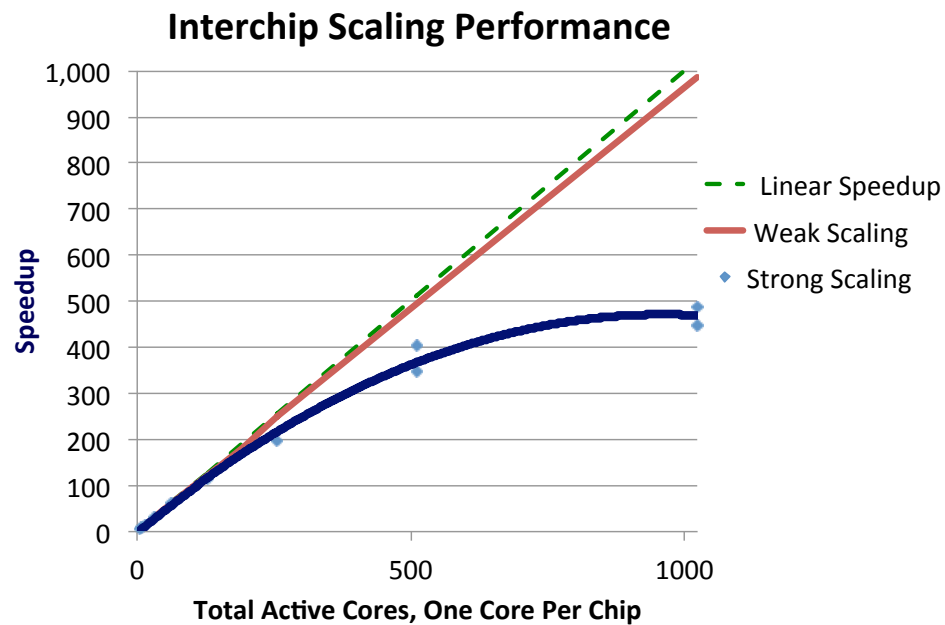
Contributions of Multicore Chips to TOP500 Rmax: 1993-2010



The Problem: Multicore Scalability

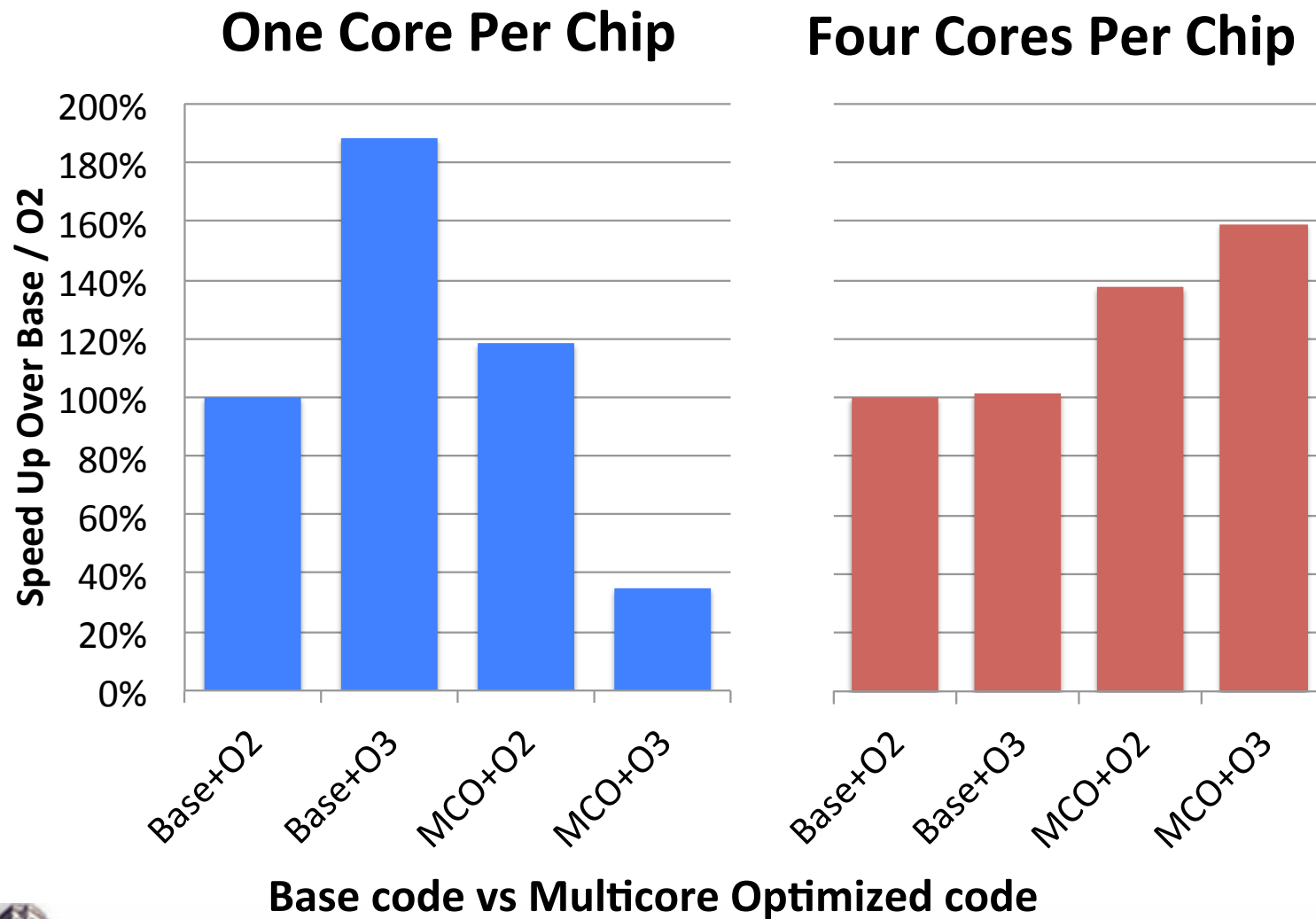


The Problem: Multicore Scalability





Optimizations Differ in Multicore





Paper Contributions

- Studies multicore related bottlenecks
- Identifies performance measurement challenges unique to multicore systems
- Presents systematic approach to multicore performance analysis
- Demonstrates principles of optimization





Talk Outline

- Introduction
- **Approach: An HPC Case Study**
- Multicore Measurement Issues
- Optimization Example
- Conclusion





Approach: An HPC Case Study

- Examine a real HPC application
 - Major functions add variety
- What is a typical HPC application?
 - Many exhibit low arithmetic intensity
 - Typical of explicit / iterative solvers, stencils
 - Finite volume / elements / differences
 - CFD, Molecular dynamics, particle simulations, graph search, Sparse MM, etc.





Approach: An HPC Case Study

- Application: HOMME

- High Order Method Modeling Environment
- 3-D Atmospheric Simulation from NCAR
- Required for NSF acceptance testing
- Excellent scaling, highly optimized
- Arithmetic Intensity typical of stencil codes

- Supercomputers:

- Ranger – 62,976 cores, 579 Teraflops
 - 2.3 GHz quad core AMD Barcelona chips
- Longhorn – 2,048 cores + 512 GPUs
 - 2.5 GHz quad core Intel Nehalem-EP chips





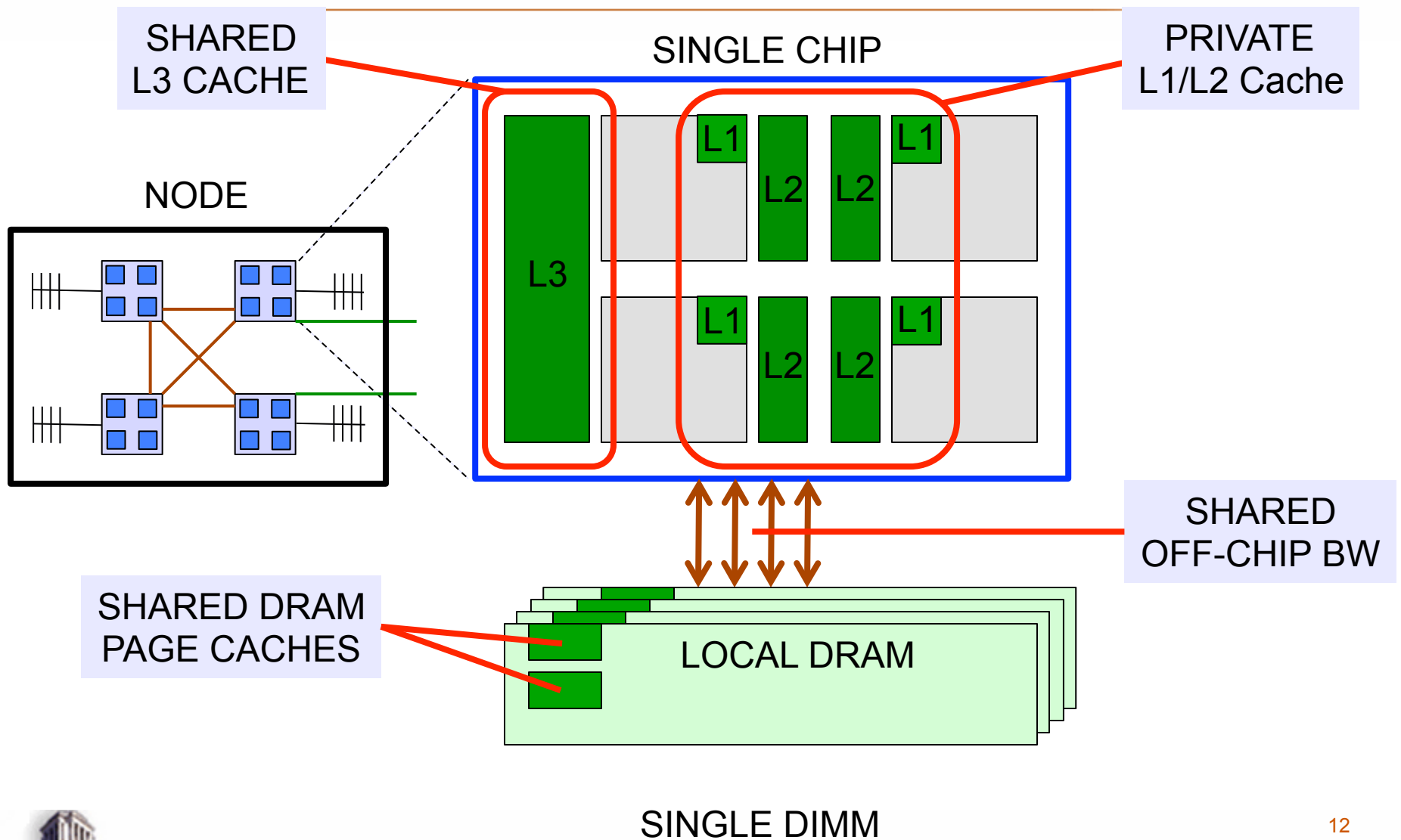
Talk Outline

- Introduction
- Approach: An HPC Case Study
- **Multicore Measurement Issues**
- Optimization Example
- Conclusion



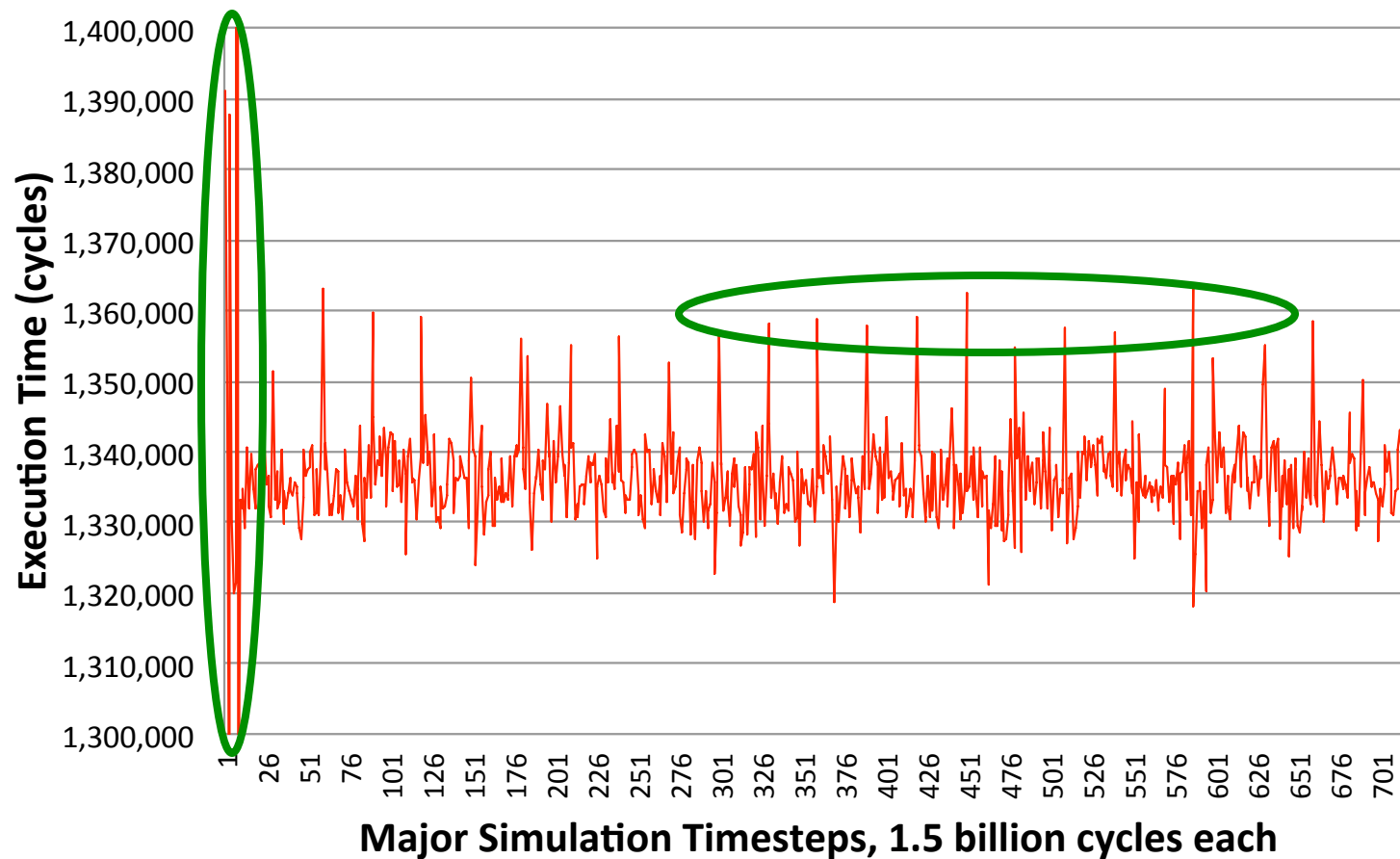


Multicore Performance Bottlenecks



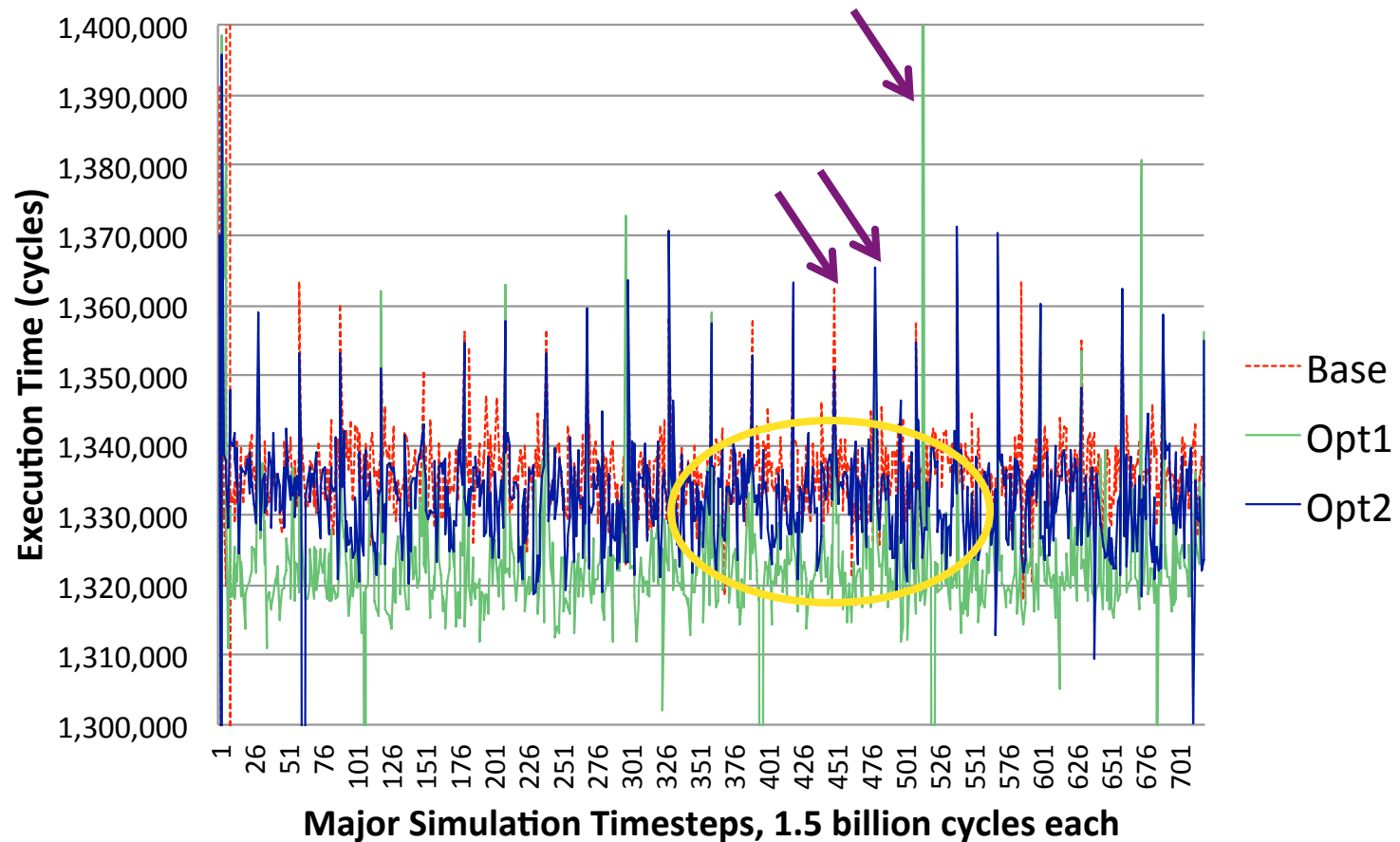
Disturbances Persist Longer

Function Performance Over Time



Measurement Implications

Function Performance Over Time





Measurements Must Be Lightweight

Action	Cycles
Read Counter	9
Read Four Counters	30
Call Function	40
PAPI READ	400
System Call	5,000
TLB Page Initialization	25,000

Function Duration	Calls Per Second	% Exec Time
2,000 cycles or less	100,000	20%
2,000 to 10,000 cycles	20,000	10%
10K to 200K cycles	1,600	15%
200K to 1M cycles	200	15%
1M to 10M cycles	-	0%
10M or more cycle	4	35%

Duration of major HOMME functions





Multicore Measurement Issues

- Performance issues in shared memory system
 - Context Sensitive
 - Nondeterministic
 - Highly non local
- Measurement disturbance is significant
 - Accessing memory or delaying core
 - Hard to “bracket” measurement effects
 - Disturbances can last billions of cycles
 - Bottlenecks can be “bursty”
- Conclusion – need multiple tools



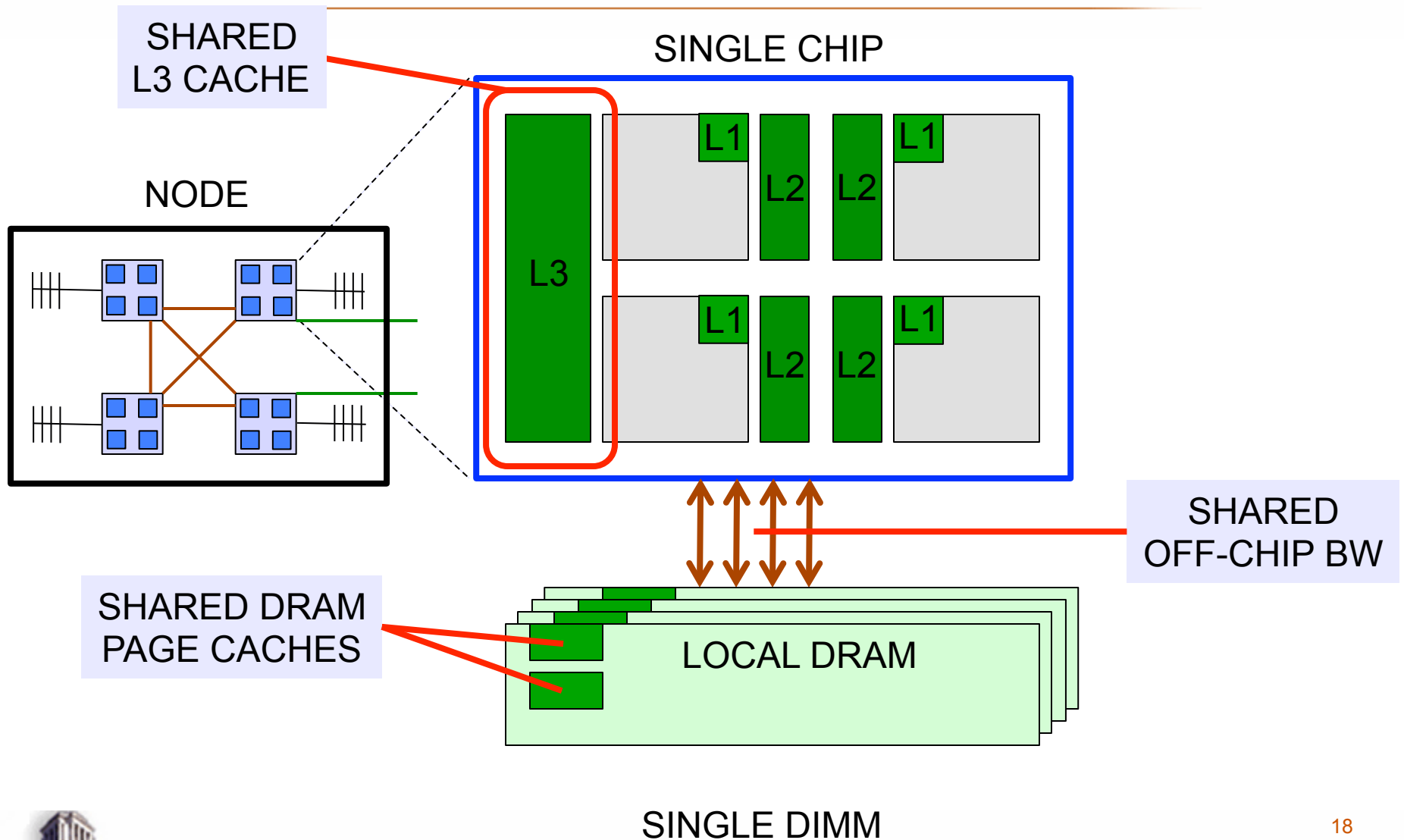


Talk Outline

- Introduction
- Approach: An HPC Case Study
- Multicore Measurement Issues
- **Optimization Example**
- Conclusion



Multicore Performance Bottlenecks





Measurement Approach

- Find important functions
- Compare performance counters at min/max core density
- Identify key multicore bottleneck:
 - L3 capacity – L3 miss rates increase with density
 - Off-chip BW – BW usage at min density greater than share
 - DRAM contention – DRAM page miss rates increase with density
- For small and medium functions, follow up with light weight / temporal measurements





Important HOMME Loop

```
do k=1,nlev
  do j=1,nv
    do i=1,nv
      T(i,j,k,n0) = T(i,j,k,n0) + smooth*(T(i,j,k,nm1) &
        - 2.0D0*T(i,j,k,n0) + T(i,j,k,np1))
      v(i,j,1,k,n0) = v(i,j,1,k,n0) + smooth*(v(i,j,1,k,nm1) &
        - 2.0D0*v(i,j,1,k,n0) + v(i,j,1,k,np1))
      v(i,j,2,k,n0) = v(i,j,2,k,n0) + smooth*(v(i,j,2,k,nm1) &
        - 2.0D0*v(i,j,2,k,n0) + v(i,j,2,k,np1))
      div(i,j,k,n0) = div(i,j,k,n0) + smooth*(div(i,j,k,nm1) &
        - 2.0D0*div(i,j,k,n0) + div(i,j,k,np1))
    end do
  end do
end do
```



Apply Microfission (First Line)

```
do k=1,nlev
  do j=1,nv      ! Load T(i,j,k,n0) into cache
    do i=1,nv    ! Block across all loops in T if won't fit
      T(i,j,k,n0) = (1.0 - 2.0*smooth)*T(i,j,k,n0)
    end do
  end do
end do

do k=1,nlev
  do j=1,nv
    do i=1,nv
      T(i,j,k,n0) = T(i,j,k,n0) + smooth*T(i,j,k,nm1)
    end do
  end do
end do

do k=1,nlev
  do j=1,nv
    do i=1,nv
      T(i,j,k,n0) = T(i,j,k,n0) + smooth*T(i,j,k,np1)
    end do
  end do
end do
```





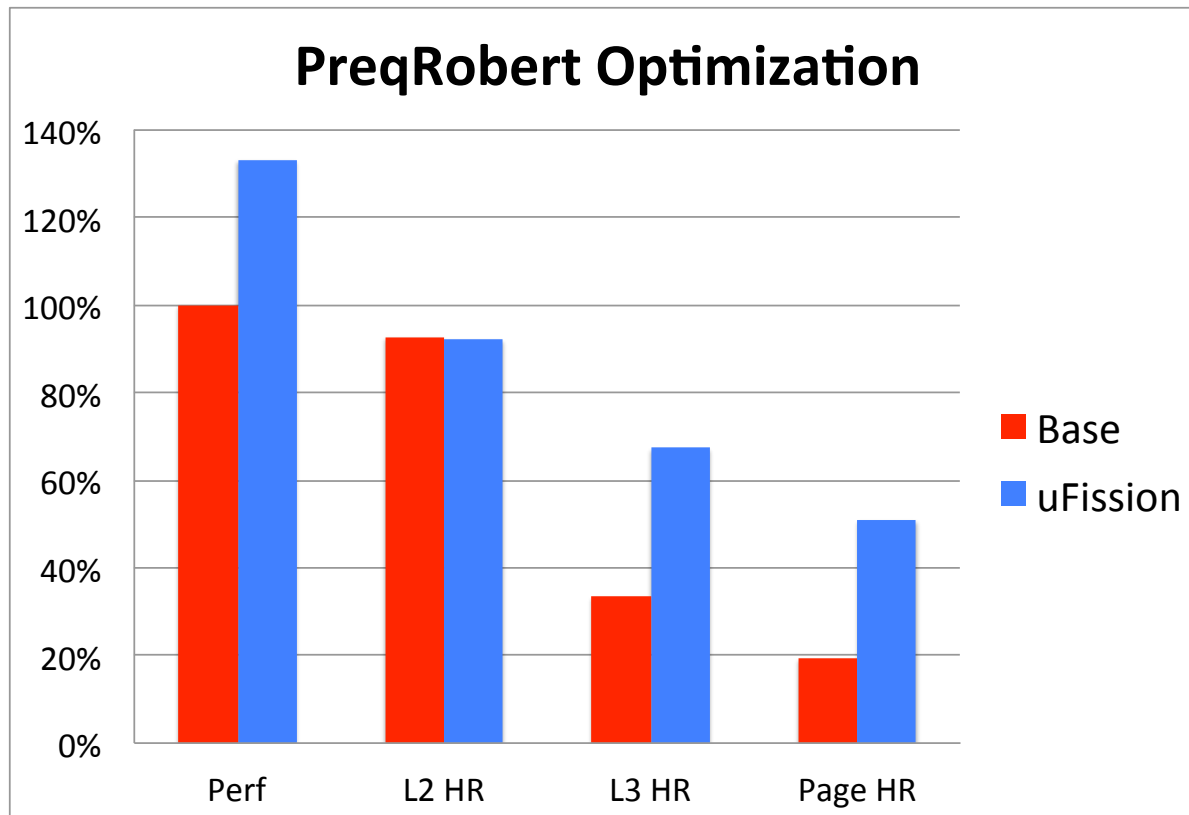
Loop Microfission

- Local, context free optimization
- Each array processed independently
 - Add high-level blocking to fit cache
- Reduces total DRAM banks accessed
 - Statistically reduces DRAM page miss rate
- Reduces instantaneous working set size
 - Helps with L3 capacity and off-chip BW





Microfission Results





Talk Outline

- Introduction
- Approach: An HPC Case Study
- Multicore Measurement Issues
- Optimization Example
- **Conclusion**





Summary and Conclusions

- HPC scalability must include multicore
 - Not well understood
 - Requires new analysis and measurement techniques
 - Optimizations differ from single-core
- Microfission is just one example
 - Multicore locality optimization for shared caches
 - Improves performance by 35%





Future Work

- Expect multicore observations apply to other HPC applications with low arithmetic intensity
 - Irregular parallel applications: Adaptive meshes, heterogeneous workloads
 - Irregular blocking applications: graph traversal
- Wider range of multicore (memory-focused) optimizations
 - Recomputation
 - Relocating Data
 - Temporary storage reduction
 - Structural changes





Thank You

- Any Questions?

